Informatics II, Spring 2026, Solution 2

Task 1

1.1 The algorithm `RecursiveAlgo` is a recursive function that checks whether a given string is a palindrome. For the input `"racecar"`, the output is `true`.

- **Base cases/terminating conditions:** If the indices meet (`s == e`), the string is trivially a palindrome and the function returns `true`. If the characters at positions `s` and `e` differ, the function immediately returns `false`. Lastly, if the indices pass each other (`s > e`), the word is also a palindrome and the function returns `true`.
- **Recursive step:** If the characters at positions `s` and `e` are equal, the function calls itself with updated indices `s + 1` and `e - 1`.
- **Progress and termination:** In each recursive call, the distance between `s` and `e` is reduced. Therefore, the recursion makes progress toward the base case and will eventually terminate.

1.2 $O(n)$

Task 2

2.1 Output: 00101

2.2 $20\%2 = 0$; `printf(0); printRec(20)`
↪ $10\%2 = 0$; `printf(0); printRec(10)`
↪ $5\%2 = 1$; `printf(1); printRec(5)`
↪ $2\%2 = 0$; `printf(0); printRec(2)`
↪ $1\%2 = 1$; `printf(1); printRec(1)`
↪ `printRec(0)`
terminating condition

2.3 Answer: Swap lines 6 and 8

```
1 void printRec(int n)
2 {
3   if (n == 0)
4     return;
5
6   printRec(n/2);
7
8   printf("%d", n%2);
9 }
```

Task 3

3.1 If there is only one disk, the problem is trivial: the disk can be moved directly from A to B.

For two disks, the larger disk cannot be moved immediately because the smaller disk is on top of it. Therefore, the smaller disk must first be moved to the remaining/auxiliary peg. Then the larger disk can be moved to the target peg. Finally, the smaller disk is moved back from the auxiliary peg onto the larger disk.

This idea can be generalized. To move n disks from A to B using C as auxiliary storage, the problem is decomposed into three steps:

- First, move the top $n-1$ disks from A to C.
- Then, move the remaining largest disk from A to B.
- Finally, move the $n-1$ disks from C onto B.

Then the same structure can be applied to move the upper $n-1$ from A to C (this time with B as the auxiliary) and so on. Always the same story until you reach the base case of one disk, where you can simply move it.

3.2

```
void Hanoi(int n, int A, int B, int C) {
  if (n == 0) return;
  Hanoi(n-1, A, C, B);
  p[B] = p[B] * 10 + p[A] % 10;
  p[A] = p[A] / 10;
  M++;
  print_pegs();
  Hanoi(n-1, C, B, A);
}
```

Task 4

```
void draw_pyramid(const int A[], int n, int level) {
   // base case: nothing to print / reduce
   if (n < 1) {
      return;
   }

   // recursive case: build reduced array of length n-1, then recurse
   int t[n - 1];
   for (int i = 0; i < n - 1; i++) {
      t[i] = A[i] + A[i + 1];
   }

   draw_pyramid(t, n - 1, level + 1);

   // print AFTER recursion so that top of pyramid appears first (top-down)
   print_level(A, n, level);
}
```